

Block 00b retrospective: why acceptance took seven reviews and many handoffs

Scope: PR #856, 2026-09-23 to 2026-09-25, from the Codex hardening session through head fbcc76672. This is a blameless record. "Agent" means Codex or Claude; both made the mistakes below.

What it cost

Measure	Count
Copilot reviews on #856	7 (all Lite effort)
Inline findings	41: 20, 6, 5, 5, 1, 1, 3
Verdicts	5 × Changes recommended, 2 × Needs a closer look, 0 clean
Red hosted CI runs	2 of 12 (35948487332 test miscount, 36083577235 cppcheck)
Owner turns in the final session	10, of which 6 were avoidable (see Handoffs)
Local runs lost to a silently wedged Docker	about 2 hours

In the last three reviews, **every finding that needed a code change (6 of 6) was a defect 00b introduced**, not an inherited one. The review loop was mostly catching our own new code.

Root causes, in order of cost

R1. A broad integration PR was sent to a sampling reviewer

#856 is a recorded exception to the SOP's small-unit rule, and the SOP also says: "do not substitute a broad product diff" for small final units. Copilot Lite behaves like a sampler: on this large diff, each round reported defects in different files, and rarely revisited earlier ones. That is why the counts were 20, 6, 5, 5, 1, 1, 3 rather than falling to zero: each round surveyed a different corner (report gate, recovery cipher, Ripple, protocol options) instead of re-checking fixes. **The number of rounds is set by PR size, not by fix quality.**

R2. 00b's new code was not self-reviewed for the classes Copilot keeps finding

All of the late findings belong to recurring classes. Each was findable with a targeted check before any review was requested:

Class	00b instance	Mechanical prevention
Comment states an invariant the code no longer keeps	LAST_ERROR moved while the comment still described the old contract; recovery delete path documented "session cipher is fixed" when it is not. (This is the same pattern as the 7.15 a56fb3e88 hardening revert.)	Encode invariants as _Static_assert or a test; a comment is not evidence. Self-review question: <i>which comment in this diff states a fact, and what proves it?</i>
Show, then validate	EIP-712 address/bytes values shown to the user before the encoder rejected them	Consent-path rule: validate → display → hash. Regression tests assert that a queued "No" survives malformed input.
Pointer formed past the buffer	Ripple memo copy *buf + n <= end	Subtraction-only bounds (has_room); grep for + .* [<>]=? end in review.
Duplicate config	Solana LUT options declared twice (plus an inherited Osmosis duplicate)	scripts/preflight.sh duplicate-options check.
A gate trusts free text	Release report took capability waivers from skip-reason text, and matched required cases by bare suffix	Gates bind to checked-in ledgers and exact identities, never to artifact prose.
Dead code after a new guard	walkVals == 0 unreachable after the shape check (CI red)	cppcheck in CI's version before every push (preflight.sh).

R3. Local "qualification" ran a different configuration than CI

- **Dirty pinned dependency.** Codex's local host runs used an uncommitted python-keepkey test edit. The emulator timer fix passed locally only because of that edit. With the pinned pyk, as CI runs it, bitcoin-only failed. The SOP already requires "pinned Docker, using CI's exact entrypoint"; nothing enforced it.
- **Wrong analyzer.** The Homebrew cppcheck (2.20) does not reproduce CI's cppcheck (2.13): a control run missed the finding CI raised. A clean local run was therefore meaningless, and that went unnoticed until a control was run.
- **Emulated architecture.** Local jobs ran linux/amd64 images under emulation on an arm64 host: a 6-minute CI job took over an hour and wedged Docker.
- **Unpinned CI tool.** CI installs cppcheck from apt on ubuntu-latest, which CI's own notice says moves to Ubuntu 26 on 2026-10-19. The analyzer, and its findings, will change with no code change.

R4. Checks were skipped after the last edit

The SOP says to "run format/build checks before review so mechanical nits do not consume rounds". The cppcheck failure happened because static analysis ran before the final source edits and not after them. This is a sequencing failure, not a knowledge gap: the rule existed.

R5. Tooling errors in the agent's own scripts

- The review poll read only the first page of reviews (30 of more than 40) and reported "no review" while one existed. The copilot-review skill's own snippet has the same bug (no --paginate).
- A zsh reply loop used set -- \$pair, which zsh does not word-split; nothing was posted. This was caught only because the thread count was verified afterwards.
- The first version of preflight.sh reported "all passed" when its cppcheck container crashed. It now fails closed and was tested against a control.

Common thread: **every check needs a negative control before its silence counts as a pass.**

R6. Handoff from a previous agent without a handoff record

The session began from a pasted Codex transcript and a working tree with 11 uncommitted files and a dirty submodule. There was no record of which checks had run against which tree. Reconstructing that state took the first hour and led directly to R3's dirty-pyk problem.

Handoffs (owner turns in the final session)

Turn	Needed?	Why it happened	Prevention
Paste Codex transcript	Yes	New session	Committed WIP plus a handoff note (R6)
"Update 00b, is it ready?"	Yes	Owner request	—
Timer fix vs pyk pin	Yes	Cross-repo pin policy is an owner decision	—
"status?"	No	A local job hung 2 h with no watchdog	Timeout every local job; check Docker health first; report within 10 minutes of a stall
"just use GitHub Actions"	No	No standing fallback when Docker fails	SOP: on a Docker failure, qualify on hosted CI and say so
"start it" (review 5)	No*	Every Copilot request needs explicit authorization	Standing authorization with a budget (owner decision below)
"always fix after reports"	No	Agent asked before fixing findings	Now standing: fix, audit, resolve, CI without asking
"one last run" (review 6)	No*	As above	As above
"approve one last call" (review 7)	No*	As above	As above

* These are avoidable only if the owner grants the standing authorization proposed below. Until then, the SOP correctly requires asking.

Changes made in this unit

1. `scripts/preflight.sh` mirrors CI Stage-1 locally and fails closed:

- whitespace
- submodules pinned and clean
- duplicate nanopb options
- clang-format 20
- cppcheck with the exact arguments extracted from `ci.yml`, Ubuntu 24.04's build, run natively
- report-gate tests
- actionlint

Verified so far: every check except cppcheck passes on the 00b head. With Docker down, the cppcheck step fails closed ("docker unresponsive") instead of passing; the first draft passed silently when its container crashed. **Still owed:** the cppcheck step must pass on the 00b head and catch the fd4a2d5ce control with CI's exact `knownConditionTrueFalse`. Docker Desktop returned HTTP 500 on `_ping` throughout this session, so this run is pending a Docker restart.

2. **static-analysis runs on ubuntu-24.04** instead of `ubuntu-latest`, so the analyzer version cannot drift silently. Upgrading becomes a deliberate change with its own findings triage.

3. **SOP amendment** (REHEARSAL-SOP.md, "Pre-push gate, self-review classes and agent handoff"): make preflight mandatory before every push; add the self-review class table above; require a negative control for any new check; add a Docker watchdog and hosted fallback; define the handoff record.

Owner decision needed (one)

To remove the review-approval handoffs without an unbounded loop: **grant standing authorization for up to 3 Copilot rounds per frozen block, at Balanced effort**. After each round the agent fixes, audits, resolves and qualifies on its own. The loop stops early on a delivered review with zero actionable findings. It escalates to the owner after round 3, or on any finding it would decline. Balanced effort is set on the PR's reviewer settings (UI or org setting); Lite has been used for all 7 rounds so far.

The larger lever is R1: future blocks should be small units, as the SOP already says. A 3-round budget on a small unit converges; on a #856-sized diff it will not.